



instituto tecnológico de saltillo

Programación lógica y funcional

5:00 a 6:00pm

Página web de perritos

Miembros del equipo:

Celeste Sughey Hernández navarro

/ROL. Frontend/DBA

Dair Antonio Riojas Quintero

/ROL. Backend/DBA

1. Requisitos previos

Se necesita Windows 10/11 con JDK 17, Oracle Database 10g Express Edición y el driver JDBC ojdbc11.jar. No hay frameworks, Maven ni Docker: el servidor web es el que trae el propio JDK

Componente	Versión exacta	Para qué se usa
Sistema operativo	Windows 10 / 11 de 64 bits	Máquina que hace de servidor
Lenguaje	Java 17 (código compilado con -- release 17)	Backend
Runtime / compilador	Eclipse Temurin JDK 17 (17.0.x); también probado con JDK 21.0.4	javac y java
Manejador de base de datos	Oracle Database 10g Express Edition 10.2.0.1	Tablas calle, color, foto, perro
Driver JDBC	Oracle ojdbc11.jar (incluido en lib/)	Conexión Java ↔ Oracle
Cliente SQL	SQL*Plus 10.2 (viene con Oracle XE)	Crear usuario, tablas y datos
Navegador	Chrome o Safari actuales (PC y celular)	Frontend
Túnel público (opcional)	cloudflared 2026.9.3	URL https para celulares fuera de la red

El frontend es HTML, CSS y JavaScript sin librerías, así que no requiere Node.js ni instalación aparte.

2. Pasos de instalación

Son ocho pasos en este orden; todos los comandos van en CMD (Símbolo del sistema).

2.1. Instalar el JDK 17 y cerrar/abrir CMD para que tome el PATH:

```
winget install --id EclipseAdoptium.Temurin.17.JDK
```

2.2. Verificar que Java quedó instalado (ambos deben decir 17):

```
java -version javac -version
```

2.3. Instalar Oracle 10g XE con su instalador (OracleXE.exe u OracleXEUniv.exe). Durante la instalación se define la contraseña de SYSTEM ; anótala. Deja el puerto 1521.

2.4. Descomprimir perros-dair.zip y entrar a la carpeta del proyecto (la que contiene config.properties):

```
cd %USERPROFILE%\Downloads\registro-perros\registro-perros
```

2.5. Confirmar que el driver está en su lugar:

```
dir lib\ojdbc11.jar
```

2.6. Crear la base de datos y cargar datos (sección 3).

2.7. Compilar el backend:

```
javac -encoding UTF-8 -cp "lib/ojdbc11.jar" -d out src/com/perros/*.java
```

2.8. (Opcional, para URL pública) Instalar el túnel:

```
winget install --id Cloudflare.cloudflare
```

3. Base de datos, catálogos y datos de prueba

Se crea el usuario perrito , se corren las tablas de sql/schema.sql y luego un script con los catálogos (10 colores, 5 calles) y 3 perros de prueba.

3.1 Crear el usuario (como SYSTEM).

Primero se fija la codificación para que acentos como "Café" se guarden igual que desde la página:

```
set NLS_LANG=SPANISH_MEXICO.AL32UTF8 sqlplus  
system/TU_PASSWORD_SYSTEM@localhost:1521/XE
```

```
CREATE USER perrito IDENTIFIED BY guau DEFAULT TABLESPACE USERS QUOTA  
UNLIMITED ON USERS; GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE,  
CREATE TRIGGER TO perrito; EXIT;
```

3.2 Crear las tablas. Desde la carpeta del proyecto:

```
sqlplus perrito/guau@localhost:1521/XE @sql/schema.sql
```

Crea las tablas calle , color , foto y perro , con una secuencia y un trigger por tabla para los IDs (10g no tiene columnas IDENTITY).

3.3 Cargar catálogos y datos de prueba.

Guardar el siguiente script como sql/datos_prueba.sql (codificación UTF-8):

```
-- Catálogo de colores (los mismos del formulario)
```

```
INSERT INTO color (color) VALUES ('Negro');
```

```
INSERT INTO color (color) VALUES ('Blanco');
```

```
INSERT INTO color (color) VALUES ('Café');
```

```
INSERT INTO color (color) VALUES ('Dorado');
```

```
INSERT INTO color (color) VALUES ('Gris');
```

```
INSERT INTO color (color) VALUES ('Manchado (blanco y negro)');
```

```
INSERT INTO color (color) VALUES ('Atigrado');
```

```
INSERT INTO color (color) VALUES ('Canela');
```

```
INSERT INTO color (color) VALUES ('Crema');
```

```
INSERT INTO color (color) VALUES ('Tricolor');
```

-- Catálogo de calles (Saltillo)

INSERT INTO calle (nom_calle) VALUES ('Calle Victoria');

INSERT INTO calle (nom_calle) VALUES ('Calle Allende');

INSERT INTO calle (nom_calle) VALUES ('Calle Hidalgo');

INSERT INTO calle (nom_calle) VALUES ('Blvd. Venustiano Carranza');

INSERT INTO calle (nom_calle) VALUES ('Blvd. Eulalio Gutierrez');

-- Fotos de prueba (copiar 3 imágenes .jpg a la carpeta fotos/ con estos nombres)

INSERT INTO foto (ruta_foto) VALUES ('fotos/prueba_1.jpg');

INSERT INTO foto (ruta_foto) VALUES ('fotos/prueba_2.jpg');

INSERT INTO foto (ruta_foto) VALUES ('fotos/prueba_3.jpg');

-- Perros de prueba

INSERT INTO perro (nombre, raza, calle_idcalle, entre_calle_idcalle1,
entre_calle1_idcalle2, color_idcolor, seg_color_idcolor1, foto_idfoto, latitud, longitud)
VALUES ('Firulais', 'Mestizo (criollo)',

(SELECT idcalle FROM calle WHERE nom_calle = 'Calle Victoria'),

(SELECT idcalle FROM calle WHERE nom_calle = 'Calle Allende'),

(SELECT idcalle FROM calle WHERE nom_calle = 'Calle Hidalgo'),

(SELECT idcolor FROM color WHERE color = 'Negro'),

(SELECT idcolor FROM color WHERE color = 'Blanco'),

(SELECT idfoto FROM foto WHERE ruta_foto = 'fotos/prueba_1.jpg'),

25.4232, -101.0053);

INSERT INTO perro (nombre, raza, calle_idcalle, color_idcolor, foto_idfoto, latitud,
longitud)

VALUES ('Canelita', 'Chihuahua',

(SELECT idcalle FROM calle WHERE nom_calle = 'Blvd. Venustiano Carranza'),

(SELECT idcolor FROM color WHERE color = 'Canela'),

(SELECT idfoto FROM foto WHERE ruta_foto = 'fotos/prueba_2.jpg'),

25.4450, -100.9860);

INSERT INTO perro (nombre, raza, calle_idcalle, color_idcolor, foto_idfoto, latitud, longitud)

VALUES ('Manchas', 'Labrador',

(SELECT idcalle FROM calle WHERE nom_calle = 'Blvd. Eulalio Gutierrez'),

(SELECT idcolor FROM color WHERE color = 'Manchado (blanco y negro)'),

(SELECT idfoto FROM foto WHERE ruta_foto = 'fotos/prueba_3.jpg'),

25.4070, -100.9700);

COMMIT;

Ejecutarlo:

sqlplus perrito/guau@localhost:1521/XE @sql/datos_prueba.sql

3.4 Verificar. Debe dar 10, 5 y 3:

SELECT COUNT(*) FROM color;

SELECT COUNT(*) FROM calle;

SELECT COUNT(*) FROM perro;

Las calles y colores que se escriban después desde la página se agregan solos al catálogo si no existen.

4. Configuración

Toda la configuración está en config.properties , en la raíz del proyecto; hay que definir 5 variables de base de datos y 2 del servidor. El programa lo busca en la carpeta desde donde se ejecuta java.

Variable	Qué es	Ejemplo	Valor si no se define
db.host	Equipo donde corre Oracle	localhost	Obligatoria
db.port	Puerto del listener de Oracle	1521	Obligatoria
db.service	Nombre de servicio de Oracle (en 10g es XE)	XE	Obligatoria
db.user	Usuario dueño de las tablas	perrito	Obligatoria
db.password	Contraseña de ese usuario	guau	Obligatoria
server.port	Puerto donde escucha la página y la API	8081	8080
fotos.carpeta	Carpeta donde se guardan las fotos subidas (se crea sola)	foto	foto

Archivo completo de ejemplo:

db.host=localhost

db.port=1521

db.service=XE

db.user=perrito

db.password=guau

server.port=8081

fotos.carpeta=fotos

Con estos datos el programa arma la cadena JDBC
jdbc:oracle:thin:@//localhost:1521/XE .

5. Ejecutar backend y frontend

Backend y frontend arrancan juntos con un solo comando: el mismo proceso Java sirve la API y los archivos de la carpeta web/, en <http://localhost:8081>.

5.1. Asegurarse de que Oracle esté encendido (como administrador):

```
net start OracleServiceXE
```

```
net start OracleXETNSListener
```

5.2. Desde la carpeta del proyecto, arrancar el servidor:

```
java -cp "out;lib/ojdbc11.jar;." Com.perros.Main
```

En Mac/Linux el separador es : en lugar de ;.

5.3. La consola muestra "Servidor iniciado correctamente" y las direcciones.

Dejar esa ventana abierta.

Pieza	URL
Frontend (página)	http://localhost:8081
Backend (API)	http://localhost:8081/api/... (ver sección 7)
Fotos subidas	http://localhost:8081/fotos/
Desde otro equipo de la red	http://:8081 (sección 6)
Desde internet (opcional)	https://.trycloudflare.com (sección 6.2)

Para detener el servidor: Ctrl + C en la ventana de Java.

6. Probarlo en el celular

En la misma red WiFi el celular entra con la IP de la PC y el puerto 8081; la cámara funciona, pero la ubicación automática solo funciona con https (sección 6.2).

6.1 Misma red WiFi 1.

Conectar el celular al mismo WiFi que la PC (no a una red de invitados, que suele aislar dispositivos).

6.1.2. En la consola de Java elegir la IP de la tarjeta WiFi, normalmente la que empieza con 192.168.1. o 192.168.0. . Ignorar 192.168.56.x (VirtualBox) y (WSL/Hyper-V).

Ejemplo: `http://192.168.1.71:8081` . 172.x.x.x

6.1.3. Si Windows pregunta por el firewall al arrancar Java, permitir en Redes privadas. Si no preguntó, abrir el puerto (CMD como administrador):

```
netsh advfirewall firewall add rule name="Perros 8081" dir=in action=allow  
protocol=TCP localport=8081
```

6.1.4. Abrir esa dirección en Chrome o Safari del celular.

6.1.5. Prueba completa: tocar "Foto del perro" (abre la cámara trasera), llenar nombre, calle y color principal, y presionar Registrar perro. Debe decir "Registro procesado." y el perro aparece en la lista y en el mapa.

6.1.6. Como en `http://` el navegador no da la ubicación, escribir latitud y longitud a mano (por ejemplo 25.4232 y -101.0053) o usar la URL https de 6.2.

6.2 Con datos móviles o desde otra red (URL pública)

Con el servidor Java corriendo, abrir otra ventana de CMD y ejecutar:

```
cloudflared tunnel --url http://localhost:8081
```

En la salida aparece una dirección `https://.trycloudflare.com`. Esa URL funciona desde cualquier celular o laptop y, por ser https, el navegador sí pide la ubicación. La URL cambia cada vez que se abre el túnel y se cae si se cierra la ventana o la PC se suspende; probarla antes de presentar con el WiFi del celular apagado.

7. Endpoints de la API

La API tiene 2 endpoints (registrar y listar) más 2 rutas que sirven archivos; todos están definidos en `src/com/perros/Main.java`.

Método	Ruta	Recibe	Respuesta	Handler
POST	/api/registrar	multipart/form-data: foto (archivo), nombre, calle, colorPrincipal (obligatorios); raza, entreCalle1, entreCalle2, colorSecundario, colorTerciario, latitud, longitud (opcionales)	200 {"ok":true} (también si era duplicado) · 400 si falta un obligatorio · 405 si no es POST · 500 error de BD	RegistrarHandler
GET	/api/perros	Nada	200 arreglo JSON de perros, más recientes primero · 500 si falla la BD	ListaPerrosHandler
GET	/fotos/{archivo}	Nombre del archivo	200 la imagen · 404 si no existe	FotosHandler
GET	/y/{archivo}	Nada	200 index.html, style.css, app.js, imágenes · 404 si no existe	EstaticoHandler

Ejemplo de un elemento de GET /api/perros:

```
{"id":1,"nombre":"Firulais","raza":"Mestizo  
(criollo)","colorPrincipal":"Negro",  
"colorSecundario":"Blanco","colorTerciario":"","calle":"Calle Victoria",  
"entreCalle1":"Calle Allende","entreCalle2":"Calle Hidalgo","fecha":"2026 09-27",  
"lat":25.4232,"lng":-101.0053,"foto":"/fotos/prueba_1.jpg"}
```

Prueba rápida desde CMD:

curl http://localhost:8081/api/perros

curl -F nombre=Toby -F calle="Calle Victoria" -F colorPrincipal=Negro -F

foto=@perro.jpg <http://localhost:8081/api/registrar>

8. Problemas comunes

La mayoría de los errores vienen de tres causas: ejecutar desde otra carpeta, Oracle apagado o la IP equivocada en el celular. El detalle de cada error aparece en la ventana de Java.

Síntoma	Causa	Solución
'javac' no se reconoce como un comando	JDK no instalado o CMD abierto antes de instalarlo	Cerrar y abrir CMD; si sigue, reinstalar el JDK (paso 1)
FileNotFoundException : config.properties o la página da "404 No encontrado"	Se ejecutó java desde otra carpeta	cd a la carpeta que contiene config.properties, web/ y out/
No se encontro el driver de Oracle (ojdbc)	Falta lib/ojdbc11.jar o el classpath está mal	Revisar el jar; en Windows usar; y en Mac/Linux : en -cp
IO Error: The Network Adapter could not establish the connection	Oracle o su listener están apagados	net start OracleServiceXE y net start OracleXETNSListener como administrador
ORA-12505 / ORA-12514 (listener no conoce el servicio)	db.service incorrecto	Usar XE ; confirmarlo con lsnrctl status
ORA-01017: invalid username/password	Usuario o contraseña distintos a config.properties	Corregir db.user / db.password
ORA-28040: No matching authentication protocol	Driver demasiado nuevo para Oracle 10g	Usar ojdbc6.jar (11.2.0.4) en lib/ y cambiar el nombre en -cp
ORA-00942: table or view does not exist	schema.sql no se corrió o se	Correr el paso 3.2 como perrito

	corrió con otro usuario	
ORA-01950: no privileges on tablespace 'USERS'	Al usuario le falta cuota	ALTER USER perrito QUOTA UNLIMITED ON USERS; como SYSTEM
BindException: Address already in use	Ya hay otra copia del servidor en el puerto 8081	Cerrar la otra ventana de Java o cambiar server.port
El celular no abre la página	IP de VirtualBox/WSL, firewall o red de invitados	Usar la IP 192.168.1.x , abrir el puerto 8081 (6.1) y conectar ambos al mismo WiFi
No aparece la ubicación automática	La página se abrió con http://	Usar la URL https del túnel o escribir latitud/longitud a mano
"Error interno del servidor" al registrar	Falla en Oracle	Leer el error completo en la consola de Java y buscarlo en esta tabla
Aparecen colores repetidos ("CafÃ©" y "Café")	Script cargado sin NLS_LANG en UTF-8	Repetir 3.1 con set NLS_LANG=SPANISH_MEXICO.AL32UTF8
El túnel da error 502 o la URL dejó de abrir	El servidor Java está cerrado o se reinició el túnel	Arrancar Java de nuevo; si se reabrió el túnel, compartir la URL nueva

9. Paradigmas

El backend es orientado a objetos, el frontend es orientado a eventos y asíncrono con partes funcionales, y la base de datos es declarativa con triggers procedurales. El registro es idempotente porque un perro con el mismo nombre, calle y color principal no se inserta dos veces y la respuesta es idéntica.

9.1 Paradigma por parte del sistema

Parte	Paradigma	Dónde está	Cómo se ve en el código
Servidor web y API	Orientado a objetos	src/com/perros/Main.java	Clases EstaticoHandler, RegistrarHandler, ListaPerrosHandler y FotosHandler implementan la interfaz HttpHandler; el servidor llama a

			handle() sin saber cuáles (polimorfismo)
Acceso a datos	Orientado a objetos (patrón DAO)	src/com/perros/PerroDAO.java, DBConnection.java	PerroDAO encapsula todo el SQL; DatosPerro y PerroInfo son objetos de datos
Lectura del formulario	Imperativo / procedural	src/com/perros/MultipartParser.java	Recorre el arreglo de bytes con ciclos e índices para separar los campos y la foto
Interacción en la página	Orientado a eventos	web/app.js	addEventListener('submit'), 'change' y 'click' disparan el registro, la vista previa y los pines
Comunicación con la API	Asíncrono	web/app.js	async/await con fetch('/api/registrar') y fetch('/api/perros'); getCurrentPosition con callbacks
Lista y mapa	Funcional	web/app.js (renderMapa, mostrarInfoPin)	filter, forEach, join y funciones puras como coordsAPixeles
Estructura de la base	Declarativo	sql/schema.sql	CREATE TABLE con llaves foráneas y restricciones UNIQUE
Consultas	Declarativo	PerroDAO.java	SELECT con JOIN en listar(); se describe qué datos se quieren, no cómo buscarlos
Generación de IDs	Procedural (PL/SQL)	sql/schema.sql	Triggers trg_*_id que toman el siguiente valor de su secuencia
Vista	Declarativo	web/index.html, web/style.css	Marcado y estilos describen la página

9.2 Idempotencia del registro

Enviar el mismo perro varias veces deja un solo registro. Está en PerroDAO.registrar() y PerroDAO.yaExiste():

9.2.1. Todo el registro corre en una transacción (setAutoCommit(false)).

9.2.2. Las calles y colores se buscan antes de crearse (buscarOCrearCalle, buscarOCrearColor), comparando con UPPER(TRIM(...)). Las restricciones UNIQUE (uq_calle_nombre, uq_color_nombre) impiden catálogos repetidos.

9.2.3. `yaExiste()` busca un perro con el mismo nombre (sin importar mayúsculas ni espacios), misma calle y mismo color principal, apoyado en el índice `ix_perro_duplicado`.

9.2.4. Si existe, se hace `rollback()` y no se inserta la foto ni el perro.

9.2.5. `RegistrarHandler` responde 200 `{"ok":true}` en ambos casos, y la página muestra el mismo mensaje "Registro procesado.". Así, reintentar (doble clic, recarga o red lenta) produce el mismo resultado.

Límites conocidos: dos envíos exactamente simultáneos podrían pasar la revisión a la vez, porque `ix_perro_duplicado` no es único; y el archivo de la foto se guarda en disco antes de la revisión, así que un duplicado deja un archivo sin usar. Ambos se resolverían con un índice único UNIQUE (`UPPER(TRIM(nombre))`), `calle_idcalle`, `color_idcolor`) y guardando la foto después de confirmar.